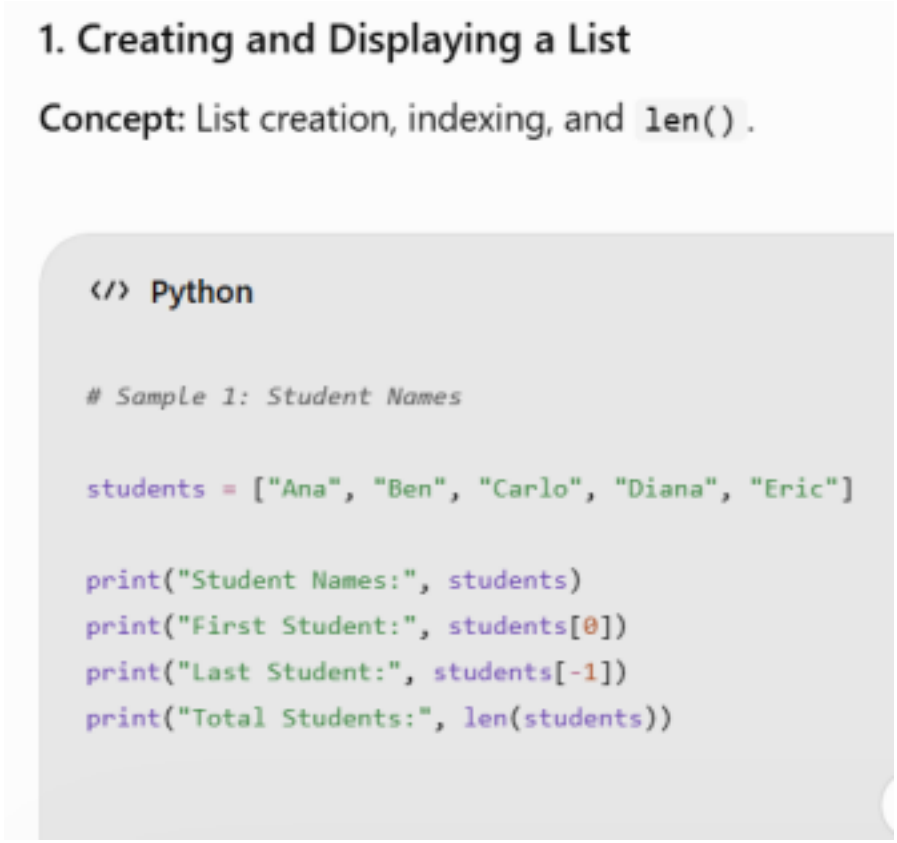
Gattoc, Mika Mclaren O.
ITC503-303i

10 Sample Python Codes: Lists and Tuples

Here are 10 sample Python programs for laboratory practice, from basic to intermediate level. Each example includes the source code and sample output.



Encoded code # 1

```
# Sample 1: Student Names

students = ["Ana", "Ben", "Carlo", "Diana", "Eric"]

print("Student Names:", students)
print("First Student:", students[0])
print("Last Student:", students[-1])
print("Total Students:", len(students))
```

```
Student Names: ['Ana', 'Ben', 'Carlo', 'Diana', 'Eric']
First Student: Ana
Last Student: Eric
Total Students: 5
```

Prepared By: VIRGINIA B. LOYOLA

2. Adding and Removing List Items

Concept: `append()`, `insert()`, `remove()`, and `pop()`.

</> Python

```python
# Sample 2: Computer Equipment

equipment = ["Monitor", "Keyboard", "Mouse"]

equipment.append("Printer")
print("After append:", equipment)

equipment.insert(1, "CPU")
print("After insert:", equipment)

equipment.remove("Mouse")
print("After remove:", equipment)

equipment.pop()
print("After pop:", equipment)
```

Encoded code # 2

```
# Sample 2: Computer Equipment

equipment = ["Monitor", "Keyboard", "Mouse"]

equipment.append("Printer")
print("After append:", equipment)

equipment.insert(1, "CPU")
print("After insert:", equipment)

equipment.remove("Mouse")
print("After remove:", equipment)

equipment.pop()
print("After pop:", equipment)
```

```
After append: ['Monitor', 'Keyboard', 'Mouse', 'Printer']
After insert: ['Monitor', 'CPU', 'Keyboard', 'Mouse', 'Printer']
After remove: ['Monitor', 'CPU', 'Keyboard', 'Printer']
After pop: ['Monitor', 'CPU', 'Keyboard']
```

Prepared By: VIRGINIA B. LOYOLA

3. List Slicing

Concept: Accessing a range of list elements.

Python

```python
# Sample 3: Programming Languages

languages = [
    "Python", "Java", "C++",
    "JavaScript", "PHP", "SQL"
]

print("All Languages:", languages)
print("First Three:", languages[:3])
print("Last Three:", languages[3:])
print("Reversed:", languages[::-1])
```

Encoded code # 3

```python
# Sample 3: Programming Languages

languages = [
    "Python", "Java", "C++",
    "JavaScript", "PHP", "SQL"
]

print("All Languages:", languages)
print("First Three:", languages[:3])
print("Last Three:", languages[3:])
print("Reversed:", languages[::-1])
```

```
All Languages: ['Python', 'Java', 'C++', 'JavaScript', 'PHP', 'SQL']
First Three: ['Python', 'Java', 'C++']
Last Three: ['JavaScript', 'PHP', 'SQL']
Reversed: ['SQL', 'PHP', 'JavaScript', 'C++', 'Java', 'Python']
```

Prepared By: VIRGINIA B. LOYOLA

4. Sorting and Searching a List

Concept: `sort()`, `in`, `min()`, and `max()`.

Python

```python
# Sample 4: Quiz Scores

scores = [85, 92, 78, 90, 88]

scores.sort()

print("Ascending:", scores)
print("Descending:", sorted(scores, reverse=True))
print("Lowest Score:", min(scores))
print("Highest Score:", max(scores))

search_score = int(input("Enter a score to search: "))

if search_score in scores:
    print("Score found!")
else:
    print("Score not found.")
```

Encoded code # 4

```python
# Sample 4: Quiz Scores

scores = [85, 92, 78, 90, 88]

scores.sort()

print("Ascending:", scores)
print("Descending:", sorted(scores, reverse=True))
print("Lowest Score:", min(scores))
print("Highest Score:", max(scores))

search_score = int(input("Enter a score to search: "))

if search_score in scores:
    print("Score found!")
else:
    print("Score not found.")
```

```
Ascending: [78, 85, 88, 90, 92]
Descending: [92, 90, 88, 85, 78]
Lowest Score: 78
Highest Score: 92
Enter a score to search: 92
Score found!
```

5. Computing the Average Using a List

Concept: `sum()`, `len()`, and arithmetic operations.

Python

```python
# Sample 5: Monthly Sales

sales = [12000, 15000, 13500, 18000, 16000, 14500]

total = sum(sales)
average = total / len(sales)

print("Monthly Sales:", sales)
print("Total Sales:", total)
print("Average Sales:", round(average, 2))
print("Highest Sales:", max(sales))
print("Lowest Sales:", min(sales))
```

6. Creating and Accessing a Tuple

Concept: Tuple creation, indexing, and immutability.

Python

```python
# Sample 6: Computer Laboratory Information

computer_lab = ("CC Building", "H310", 40, "Advanced Database")

print("Laboratory Information:", computer_lab)
print("Building:", computer_lab[0])
print("Room:", computer_lab[1])
print("Computers:", computer_lab[2])
print("Subject:", computer_lab[3])
print("Number of Items:", len(computer_lab))
```

Encoded code # 5

➔

```
# Sample 5: Monthly Sales

sales = [12000, 15000, 13500, 18000, 16000, 14500]

total = sum(sales)
average = total / len(sales)

print("Monthly Sales:", sales)
print("Total Sales:", total)
print("Average Sales:", round(average, 2))
print("Highest Sales:", max(sales))
print("Lowest Sales:", min(sales))
```

```
Monthly Sales: [12000, 15000, 13500, 18000, 16000, 14500]
Total Sales: 89000
Average Sales: 14833.33
Highest Sales: 18000
Lowest Sales: 12000
```

Encoded code # 6

```python
# Sample 6: Computer Laboratory Information

computer_lab = ("CC Building", "H310", 40, "Advanced Database")

print("Laboratory Information:", computer_lab)
print("Building:", computer_lab[0])
print("Room:", computer_lab[1])
print("Computers:", computer_lab[2])
print("Subject:", computer_lab[3])
print("Number of Items:", len(computer_lab))
```

```
Laboratory Information: ('CC Building', 'H310', 40, 'Advanced Database')
Building: CC Building
Room: H310
Computers: 40
Subject: Advanced Database
Number of Items: 4
```

Prepared By: VIRGINIA B. LOYOLA

7. Tuple Methods: `count()` and `index()`

Concept: Counting occurrences and finding an element's position.

Python

```python
# Sample 7: IT Tools

tools = ("Python", "SQL", "Java", "Python", "Excel", "Python")

print("Tools:", tools)
print("Python Count:", tools.count("Python"))
print("Python First Index:", tools.index("Python"))
print("SQL First Index:", tools.index("SQL"))
```

Encoded code # 7

```python
# Sample 7: IT Tools

tools = ("Python", "SQL", "Java", "Python", "Excel", "Python")

print("Tools:", tools)
print("Python Count:", tools.count("Python"))
print("Python First Index:", tools.index("Python"))
print("SQL First Index:", tools.index("SQL"))
```

```
Tools: ('Python', 'SQL', 'Java', 'Python', 'Excel', 'Python')
Python Count: 3
Python First Index: 0
SQL First Index: 1
```

Prepared By: VIRGINIA B. LOYOLA

8. Combining and Repeating Tuples

Concept: Tuple concatenation (`+`) and repetition (`*`).

Python

```python
# Sample 8: Course Information

course1 = ("Python", "Database")
course2 = ("Networking", "Cybersecurity")

combined = course1 + course2
repeated = course1 * 2

print("Course 1:", course1)
print("Course 2:", course2)
print("Combined Tuple:", combined)
print("Repeated Tuple:", repeated)
```

Encoded code # 8

```python
# Sample 8: Course Information

course1 = ("Python", "Database")
course2 = ("Networking", "Cybersecurity")

combined = course1 + course2
repeated = course1 * 2

print("Course 1:", course1)
print("Course 2:", course2)
print("Combined Tuple:", combined)
print("Repeated Tuple:", repeated)
```

```
Course 1: ('Python', 'Database')
Course 2: ('Networking', 'Cybersecurity')
Combined Tuple: ('Python', 'Database', 'Networking', 'Cybersecurity')
Repeated Tuple: ('Python', 'Database', 'Python', 'Database')
```

Prepared By: VIRGINIA B. LOYOLA

9. Converting a List to a Tuple and Back

Concept: `tuple()` and `list()`.

Python

```
# Sample 9: Subject Enrollment

subjects = ["Statistics", "Database", "Programming"]

print("Original List:", subjects)

subject_tuple = tuple(subjects)
print("Converted Tuple:", subject_tuple)

updated_subjects = list(subject_tuple)
updated_subjects.append("Data Science")

print("Updated List:", updated_subjects)
```

Encoded code # 9

```python
# Sample 9: Subject Enrollment

subjects = ["Statistics", "Database", "Programming"]

print("Original List:", subjects)

subject_tuple = tuple(subjects)
print("Converted Tuple:", subject_tuple)

updated_subjects = list(subject_tuple)
updated_subjects.append("Data Science")

print("Updated List:", updated_subjects)
```

```
Original List: ['Statistics', 'Database', 'Programming']
Converted Tuple: ('Statistics', 'Database', 'Programming')
Updated List: ['Statistics', 'Database', 'Programming', 'Data Science']
```

Prepared By: VIRGINIA B. LOYOLA

10. Removing Duplicate Items from a List

Concept: Lists, loops, conditional statements, and `append()`.

</> Python

```python
# Sample 10: Remove Duplicate Programming Languages

data = [
    "Python", "Java", "Python",
    "C++", "Java", "SQL", "C++"
]

unique = []

for item in data:
    if item not in unique:
        unique.append(item)

print("Original List:", data)
print("Unique List:", unique)
print("Number of Unique Items:", len(unique))
```

Encoded code # 10

```python
# Sample 10: Remove Duplicate Programming Languages

data = [
    "Python", "Java", "Python",
    "C++", "Java", "SQL", "C++"
]

unique = []

for item in data:
    if item not in unique:
        unique.append(item)

print("Original List:", data)
print("Unique List:", unique)
print("Number of Unique Items:", len(unique))
```

```
Original List: ['Python', 'Java', 'Python', 'C++', 'Java', 'SQL', 'C++']
Unique List: ['Python', 'Java', 'C++', 'SQL']
Number of Unique Items: 4
```

Prepared By: VIRGINIA B. LOYOLA

10 Difficult-Level Python Sample Codes: Lists and Tuples

These examples are suitable for intermediate to advanced laboratory exercises for IT students. They combine lists, tuples, nested data structures, loops, conditional statements, sorting, searching, and data processing.

Sample 1: Remove Duplicates While Preserving Order

Concepts: Lists, loops, membership testing, conditional statements.

Problem: Remove duplicate student names without changing the order of their first appearance.

‹/› Python

```python
students = [
    "Ana", "Ben", "Carlo", "Ana",
    "Diana", "Ben", "Eric", "Carlo"
]

unique_students = []

for student in students:
    if student not in unique_students:
        unique_students.append(student)

print("Original List:", students)
print("Unique Students:", unique_students)
print("Total Unique Students:", len(unique_students))
```

Encoded code sample # 1

```python
# Difficult Sample 1: Remove Duplicates

students = [
    "Ana", "Ben", "Carlo", "Ana",
    "Diana", "Ben", "Eric", "Carlo"
]

unique_students = []

for student in students:
    if student not in unique_students:
        unique_students.append(student)

print("Original List:", students)
print("Unique Students:", unique_students)
print("Total Unique Students:", len(unique_students))
```

```
Original List: ['Ana', 'Ben', 'Carlo', 'Ana', 'Diana', 'Ben', 'Eric', 'Carlo']
Unique Students: ['Ana', 'Ben', 'Carlo', 'Diana', 'Eric']
Total Unique Students: 5
```

Prepared By: VIRGINIA B. LOYOLA

Sample 2: Find the Second-Highest Unique Score

Concepts: Lists, sets, sorting, duplicate handling.

Problem: Find the second-highest distinct quiz score, even when the highest score appears more than once.

Python

```python
scores = [85, 92, 78, 95, 92, 88, 95, 80]

unique_scores = sorted(set(scores), reverse=True)

print("Scores:", scores)
print("Ranked Unique Scores:", unique_scores)

if len(unique_scores) >= 2:
    print("Highest Score:", unique_scores[0])
    print("Second-Highest Score:", unique_scores[1])
else:
    print("Not enough distinct scores.")
```

Encoded code sample # 2

```python
# Difficult Sample 2: Second-Highest Score

scores = [85, 92, 78, 95, 92, 88, 95, 80]

unique_scores = sorted(set(scores), reverse=True)

print("Scores:", scores)
print("Ranked Unique Scores:", unique_scores)

if len(unique_scores) >= 2:
    print("Highest Score:", unique_scores[0])
    print("Second-Highest Score:", unique_scores[1])
else:
    print("Not enough distinct scores.")
```

```
Scores: [85, 92, 78, 95, 92, 88, 95, 80]
Ranked Unique Scores: [95, 92, 88, 85, 80, 78]
Highest Score: 95
Second-Highest Score: 92
```

Prepared By: VIRGINIA B. LOYOLA

Sample 3: Frequency Distribution Using a List

Concepts: Lists, tuples, loops, `count()`, conditional logic.

Problem: Count how many times each distinct programming language appears in a survey response list.

</> Python

```python
responses = [
    "Python", "Java", "Python", "C++",
    "Java", "Python", "SQL", "C++",
    "Java", "SQL", "Python"
]

frequency = []

for language in responses:
    found = False

    for i in range(len(frequency)):
        if frequency[i][0] == language:
            frequency[i] = (
                language,
                frequency[i][1] + 1
            )
            found = True
            break
```

```
    if not found:
        frequency.append((language, 1))

print("Language Frequency")

for language, count in frequency:
    print(language, ":", count)
```

Encoded code sample # 3

```python
# Difficult Sample 3: Frequency Distribution

responses = [
    "Python", "Java", "Python", "C++",
    "Java", "Python", "SQL", "C++",
    "Java", "SQL", "Python"
]

frequency = []

for language in responses:
    found = False

    for i in range(len(frequency)):
        if frequency[i][0] == language:
            frequency[i] = (
                language,
                frequency[i][1] + 1
            )
            found = True
            break

    if not found:
        frequency.append((language, 1))

print("Language Frequency")

for language, count in frequency:
    print(language, ":", count)
```

```
Language Frequency
Python : 4
Java : 3
C++ : 2
SQL : 2
```

Prepared By: VIRGINIA B. LOYOLA

Each entry in `frequency` is a tuple containing a language and its count.

Sample 4: Student Ranking System

Concepts: Nested tuples, sorting with `key`, lambda functions.

Problem: Rank students according to their average grades, from highest to lowest.

</> Python

```python
students = [
    ("Ana", (90, 85, 95)),
    ("Ben", (78, 88, 82)),
    ("Carlo", (92, 91, 89)),
    ("Diana", (85, 87, 90)),
    ("Eric", (92, 91, 89))
]
```

```
results = []

for name, grades in students:
    average = sum(grades) / len(grades)
    results.append((name, average))

results.sort(key=lambda item: (-item[1], item[0]))

print("STUDENT RANKING")

for rank, (name, average) in enumerate(results, start=1):
    print(f"{rank}. {name}: {average:.2f}")
```

Encoded code sample # 4

```python
# Difficult Sample 4: Student Ranking

students = [
    ("Ana", (90, 85, 95)),
    ("Ben", (78, 88, 82)),
    ("Carlo", (92, 91, 89)),
    ("Diana", (85, 87, 90)),
    ("Eric", (92, 91, 89))
]

results = []

for name, grades in students:
    average = sum(grades) / len(grades)
    results.append((name, average))

results.sort(key=lambda item: (-item[1], item[0]))

print("STUDENT RANKING")

for rank, (name, average) in enumerate(results, start=1):
    print(f"{rank}. {name}: {average:.2f}")
```

```
STUDENT RANKING
1. Carlo: 90.67
2. Eric: 90.67
3. Ana: 90.00
4. Diana: 87.33
5. Ben: 82.67
```

Prepared By: VIRGINIA B. LOYOLA

Sample 5: Matrix Transposition Using Nested Lists

Concepts: Nested lists, indexing, loops, list comprehensions.

Problem: Convert a matrix with three rows and four columns into a matrix with four rows and three columns.

</> Python

```python
matrix = [
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]
]

rows = len(matrix)
columns = len(matrix[0])

transposed = []
```

```
for j in range(columns):
    new_row = []

    for i in range(rows):
        new_row.append(matrix[i][j])

    transposed.append(new_row)

print("Original Matrix:")
for row in matrix:
    print(row)

print("\nTransposed Matrix:")
for row in transposed:
    print(row)
```

Encoded code sample # 5

```python
# Difficult Sample 5: Matrix Transposition
matrix = [
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]
]

rows = len(matrix)
columns = len(matrix[0])

transposed = []

for j in range(columns):
    new_row = []

    for i in range(rows):
        new_row.append(matrix[i][j])

    transposed.append(new_row)

print("Original Matrix:")
for row in matrix:
    print(row)

print("\nTransposed Matrix:")
for row in transposed:
    print(row)
```

```
Original Matrix:
[10, 20, 30, 40]
[50, 60, 70, 80]
[90, 100, 110, 120]

Transposed Matrix:
[10, 50, 90]
[20, 60, 100]
[30, 70, 110]
[40, 80, 120]
```

Prepared By: VIRGINIA B. LOYOLA

Sample 6: Common and Unique Elements Between Two Lists

Concepts: Lists, sets, membership testing, set operations.

Problem: Compare two lists of students enrolled in different IT electives.

Python

```
python_students = [
    "Ana", "Ben", "Carlo", "Diana", "Eric"
]

database_students = [
    "Ben", "Diana", "Frank", "Grace", "Eric"
]

python_set = set(python_students)
database_set = set(database_students)

common = sorted(python_set & database_set)
python_only = sorted(python_set - database_set)
database_only = sorted(database_set - python_set)
```

```
print("Enrolled in Both:", common)
print("Python Only:", python_only)
print("Database Only:", database_only)
print("Total Distinct Students:",
      len(python_set | database_set))
```

Encoded code sample # 6

```python
# Difficult Sample 6: Common and Unique Elements

python_students = [
    "Ana", "Ben", "Carlo", "Diana", "Eric"
]

database_students = [
    "Ben", "Diana", "Frank", "Grace", "Eric"
]

python_set = set(python_students)
database_set = set(database_students)

common = sorted(python_set & database_set)
python_only = sorted(python_set - database_set)
database_only = sorted(database_set - python_set)

print("Enrolled in Both:", common)
print("Python Only:", python_only)
print("Database Only:", database_only)
print("Total Distinct Students:",
      len(python_set | database_set))
```

```
Enrolled in Both: ['Ben', 'Diana', 'Eric']
Python Only: ['Ana', 'Carlo']
Database Only: ['Frank', 'Grace']
Total Distinct Students: 7
```

Prepared By: VIRGINIA B. LOYOLA

Sample 7: Rotate a List by a User-Defined Number

Concepts: List slicing, modulo operator, user input.

Problem: Rotate a list of tasks to the right by a number of positions entered by the user.

</> Python

```
tasks = [
    "Design ERD",
    "Create Tables",
    "Insert Data",
    "Test Queries",
    "Prepare Report"
]

k = int(input("Rotate right by how many positions? "))

if tasks:
    k = k % len(tasks)
    rotated = tasks[-k:] + tasks[:-k] if k else tasks[:]
else:
    rotated = []

print("Original Tasks:", tasks)
print("Rotated Tasks:", rotated)
```

Encoded code sample # 7

```python
# Difficult Sample 7: Rotate a List

tasks = [
    "Design ERD",
    "Create Tables",
    "Insert Data",
    "Test Queries",
    "Prepare Report"
]

k = int(input("Rotate right by how many positions? "))

if tasks:
    k = k % len(tasks)
    rotated = tasks[-k:] + tasks[:-k] if k else tasks[:]
else:
    rotated = []

print("Original Tasks:", tasks)
print("Rotated Tasks:", rotated)
```

```
Rotate right by how many positions? 3
Original Tasks: ['Design ERD', 'Create Tables', 'Insert Data', 'Test Queries', 'Prepare Report']
Rotated Tasks: ['Insert Data', 'Test Queries', 'Prepare Report', 'Design ERD', 'Create Tables']
```

Prepared By: VIRGINIA B. LOYOLA

Sample 8: Merge Two Lists of Records Using Tuples

Concepts: Tuples, dictionaries, loops, record matching.

Problem: Merge student information and grades using student ID as the key.

</> Python

```python
student_info = [
    (101, "Ana"),
    (102, "Ben"),
    (103, "Carlo"),
    (104, "Diana")
]

student_grades = [
    (103, 91),
    (101, 95),
    (104, 88),
    (102, 84)
]

grade_lookup = dict(student_grades)
merged_records = []
```

```
for student_id, name in student_info:
    if student_id in grade_lookup:
        merged_records.append(
            (student_id, name, grade_lookup[student_id])
        )

print("ID | Name | Grade")

for record in merged_records:
    print(record[0], "|", record[1], "|", record[2])
```

Encoded code sample # 8

```python
# Difficult Sample 8: Merge Student Records

student_info = [
    (101, "Ana"),
    (102, "Ben"),
    (103, "Carlo"),
    (104, "Diana")
]

student_grades = [
    (103, 91),
    (101, 95),
    (104, 88),
    (102, 84)
]

grade_lookup = dict(student_grades)
merged_records = []

for student_id, name in student_info:
    if student_id in grade_lookup:
        merged_records.append(
            (student_id, name, grade_lookup[student_id])
        )

print("ID | Name | Grade")

for record in merged_records:
    print(record[0], "|", record[1], "|", record[2])
```

```
ID | Name | Grade
101 | Ana | 95
102 | Ben | 84
103 | Carlo | 91
104 | Diana | 88
```

Prepared By: VIRGINIA B. LOYOLA

Sample 9: Find Pairs That Equal a Target Sum

Concepts: Nested loops, tuples, list indexing, conditional statements.

Problem: Find all unique pairs of quiz scores whose sum equals a target value.

‹/› Python

```python
numbers = [10, 15, 20, 25, 30, 35, 15]
target = 45

pairs = []

for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)):
        if numbers[i] + numbers[j] == target:
            pair = tuple(sorted((numbers[i], numbers[j])))

            if pair not in pairs:
                pairs.append(pair)

print("Numbers:", numbers)
print("Target Sum:", target)
print("Unique Pairs:", pairs)
```

Encoded code sample # 9

```python
# Difficult Sample 9: Target Sum

numbers = [10, 15, 20, 25, 30, 35, 15]
target = 45

pairs = []

for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)):
        if numbers[i] + numbers[j] == target:
            pair = tuple(sorted((numbers[i], numbers[j])))

            if pair not in pairs:
                pairs.append(pair)

print("Numbers:", numbers)
print("Target Sum:", target)
print("Unique Pairs:", pairs)
```

```
Numbers: [10, 15, 20, 25, 30, 35, 15]
Target Sum: 45
Unique Pairs: [(10, 35), (15, 30), (20, 25)]
```

Prepared By: VIRGINIA B. LOYOLA

Sample 10: Inventory Management Using Lists and Tuples

Concepts: Lists of tuples, searching, updating records, filtering, sorting.

Problem: Develop a small inventory program that calculates stock values, identifies low-stock products, and ranks products by total inventory value.

Python

```python
inventory = [
    ("Mouse", 10, 350),
    ("Keyboard", 5, 650),
    ("Monitor", 3, 5500),
    ("USB Drive", 20, 250),
    ("Headset", 4, 1200)
]

low_stock_limit = 5
total_inventory_value = 0
low_stock = []
stock_values = []
```

```
for name, quantity, price in inventory:
    value = quantity * price
    total_inventory_value += value

    stock_values.append((name, value))

    if quantity < low_stock_limit:
        low_stock.append((name, quantity))

stock_values.sort(key=lambda item: item[1], reverse=True)

print("INVENTORY REPORT")
```

Encoded code sample # 10

```python
# Difficult Sample 10: Inventory Management

inventory = [
    ("Mouse", 10, 350),
    ("Keyboard", 5, 650),
    ("Monitor", 3, 5500),
    ("USB Drive", 20, 250),
    ("Headset", 4, 1200)
]

low_stock_limit = 5
total_inventory_value = 0
low_stock = []
stock_values = []

for name, quantity, price in inventory:
    value = quantity * price
    total_inventory_value += value

    stock_values.append((name, value))

    if quantity < low_stock_limit:
        low_stock.append((name, quantity))

stock_values.sort(key=lambda item: item[1], reverse=True)

print("INVENTORY REPORT")

for name, quantity, price in inventory:
    print(
        f"{name}: Quantity={quantity}, "
        f"Price={price}, Value={quantity * price}"
    )

print("\nTotal Inventory Value:", total_inventory_value)
print("Products Below Stock Limit:", low_stock)
```

→

```
INVENTORY REPORT
Mouse: Quantity=10, Price=350, Value=3500
Keyboard: Quantity=5, Price=650, Value=3250
Monitor: Quantity=3, Price=5500, Value=16500
USB Drive: Quantity=20, Price=250, Value=5000
Headset: Quantity=4, Price=1200, Value=4800

Total Inventory Value: 33050
Products Below Stock Limit: [('Monitor', 3), ('Headset', 4)]
```

→

```
for name, quantity, price in inventory:
    print(
        f"{name}: Quantity={quantity}, "
        f"Price={price}, Value={quantity * price}"
    )

print("\nTotal Inventory Value:", total_inventory_value)
print("Products Below Stock Limit:", low_stock)
```

Summary of Python List and Tuple Operations

Operation	List	Tuple
Create	`items = [1, 2, 3]`	`items = (1, 2, 3)`
Access an item	`items[0]`	`items[0]`
Count elements	`len(items)`	`len(items)`
Count occurrences	`items.count(2)`	`items.count(2)`
Find index	`items.index(2)`	`items.index(2)`
Add elements	`append()`, `insert()`	Not directly supported
Remove elements	`remove()`, `pop()`	Not directly supported